## CLASE 28

# Django Setup

*"El framework web para perfeccionistas con deadlines"*

### Hoy aprenderemos:

- Qué es Django y por qué usarlo
- Instalar Django en entorno virtual
- Crear un proyecto Django
- Entender la estructura de archivos
- Correr el servidor de desarrollo
- Crear la primera aplicación

**Inicio del proyecto: Dashboard E-commerce**

# ¿Qué es Django?

### Django = Framework web de Python

Creado en 2005, usado por millones de sitios
Framework = Estructura + herramientas pre-construidas
**No empiezas desde cero. Ya tienes la base.**

### ¿Quién usa Django?

Instagram • Pinterest • Spotify
The Washington Post • Mozilla
NASA • Y muchos más...

### ¿Por qué se llama Django?

En honor a Django Reinhardt, guitarrista de jazz legendario de los años 30. Los creadores del framework eran fans de su música.

**Con Django, construyen en semanas lo que tomaría meses desde cero**

# ¿Por qué elegir Django?

*"Batteries included" — Baterías incluidas*

**ORM**
Base de datos sin escribir SQL

**Autenticación**
Login, registro, permisos

**Admin**
Panel automático incluido

**Seguridad**
Protección contra ataques

**Migraciones**
Control de versión de BD

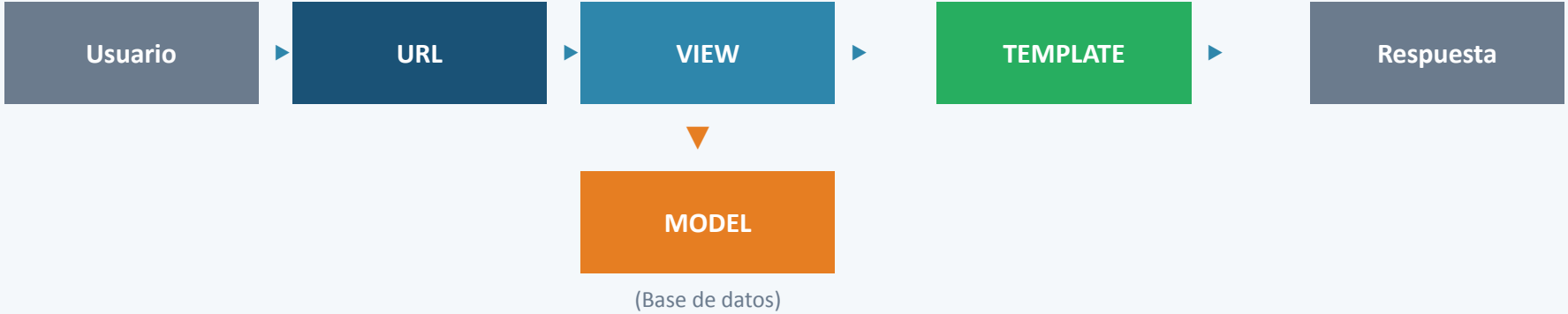
**Formularios**
Validación incluida

**Comparación de frameworks:**

Flask = Microframework (lo mínimo, tú agregas todo)
**Django = Framework completo (todo incluido)**
FastAPI = Especializado en APIs (muy rápido)

# Arquitectura MTV de Django

*MTV = Model - Template - View*




### Model (Modelo)

Define la estructura de datos.
Se conecta con la base de datos.
Es una clase de Python.

### View (Vista)

La lógica de negocio.
Conecta modelo con template.
Decide qué mostrar.

### Template (Plantilla)

El HTML que ve el usuario.
Puede incluir variables.
Contenido dinámico.

# Preparando el entorno

1. **Crear carpeta del proyecto** — Una carpeta nueva para todo
2. **Crear entorno virtual** — Cada proyecto, su propio entorno
3. **Activar el entorno** — Verificar que aparezca (venv)
4. **Instalar Django** — pip install django

**Estructura inicial:**

```
ecommerce_dashboard/
  └── venv/

  ← Entorno virtual
```

**Hagámoslo juntos paso a paso**

# >_ Crear el proyecto: startproject

```
django-admin startproject nombre_proyecto
```

### ¿Qué hace?

- Crea la estructura de carpetas
- Genera archivos de configuración
- Prepara todo para empezar a trabajar

### Analogía:

startproject = Comprar un terreno
con los cimientos listos

startapp = Construir una casa
en ese terreno

**Convención de nombres:** minúsculas, sin espacios ni guiones (usar guión bajo), descriptivo

Nuestro proyecto: **`dashboard`**

# Estructura del proyecto Django

```
ecommerce_dashboard/
├── venv/
└── dashboard/
    ├── dashboard/
    │   ├── __init__.py
    │   ├── settings.py
    │   ├── urls.py
    │   ├── asgi.py
    │   └── wsgi.py
    └── manage.py
```

**Los más importantes:**

**manage.py** Ejecutar comandos

**settings.py** Configuración

**urls.py** Definir rutas/URLs

**Sí, hay dos carpetas "dashboard"**
Es confuso pero normal.

# settings.py — Configuración

### DEBUG = True

Modo desarrollo
Muestra errores detallados
En producción debe ser False

### INSTALLED_APPS

Lista de apps activas
Aquí registramos nuestras apps
Django trae varias por defecto

### DATABASES

Configuración de base de datos
Por defecto usa SQLite
Perfecto para desarrollo

### LANGUAGE / TIME_ZONE

Idioma y zona horaria
Por defecto: inglés y UTC
Se puede cambiar fácilmente

**Por ahora no tocamos settings.py — Funciona bien con los defaults**

# Primer comando: migrate

```
python manage.py migrate
```

### ¿Qué hace migrate?

- Crea las tablas necesarias en la BD
- Django necesita tablas para usuarios, sesiones
- Ejecuta “migraciones” pendientes

Salida esperada:

```
Applying contenttypes... OK
Applying auth.0001...    OK
Applying admin.0001...   OK
Applying sessions...     OK
```

Después de ejecutar: Se crea el archivo db.sqlite3 (la base de datos) y las tablas base quedan listas

**Migraciones = Control de versiones para la base de datos**

# runserver — El momento de la verdad

```
python manage.py runserver
```

### ¿Qué hace?

- Inicia un servidor web en tu computadora
- Por defecto en http://127.0.0.1:8000
- 127.0.0.1 = localhost = tu máquina
- 8000 = puerto
- **Para detener: Ctrl + C**

Terminal:

```
Starting development server
at http://127.0.0.1:8000/

Quit the server with
CTRL-BREAK.
```

**Es servidor de DESARROLLO — No usar en producción**

**¡Corramos el servidor!**

# BREAK — 10 MINUTOS

**Hasta ahora:**

- Django instalado
- Proyecto creado
- Servidor corriendo

**Volvemos con:**

- Crear nuestra primera app
- Primera vista real

# Proyecto vs App en Django

**PROYECTO** = El sitio web completo

**APP** = Un componente/módulo

## Analogía: Un edificio

| Proyecto | App | App | App |
|---|---|---|---|
| `"dashboard"` | `"core"` | `"productos"` | `"usuarios"` |
| El edificio | Recepción | Tienda | Oficina |

### Para nuestro Dashboard E-commerce:

- core ⟶ Landing page, páginas generales
- productos ⟶ CRUD de productos
- ventas ⟶ Registro de ventas
- dashboard ⟶ Métricas y gráficos

# Crear una app: startapp

```
python manage.py startapp core
```

**¿Qué crea startapp?**

```
core/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── tests.py
├── views.py
└── migrations/
```

**Archivos importantes:**

models.py → Modelos (base de datos)
views.py → Vistas (lógica)
admin.py → Config del admin
migrations/ → Historial de BD

**Después de crear:**

**REGISTRAR en settings.py**

(Django no la detecta automáticamente)

# Nuestra primera View

**Una VIEW es una función que:**

1. Recibe una petición (request)
2. Procesa algo (opcional)
3. Devuelve una respuesta (response)

Patrón básico de una vista:

request = La petición del navegador
HttpResponse = Una respuesta simple (texto o HTML)

**Después de crear la view:**

1. Registrarla en urls.py 2. Asociarla a una URL

# Agregando más vistas

**Patrón repetible:**

1. Crear función en views.py
2. Importar en urls.py
3. Agregar path() en urlpatterns

**Ejemplo con múltiples páginas:**

| URL | Vista | Descripción |
|---|---|---|
| `/` | `home` | Página principal |
| `/about/` | `about` | Sobre nosotros |
| `/contact/` | `contact` | Contacto |

Las URLs en Django son limpias: **NO necesitan extensión (.html, .php)**

*Próxima clase: Templates (HTML real, no strings)*

# Bonus: El Admin de Django

*Django incluye un panel de administración automático*

**Crear superusuario:**

```
python manage.py createsuperuser

Username: admin
Email: admin@example.com
Password: ********
```

### ¿Qué incluye el Admin?

- Gestión de usuarios
- Grupos y permisos
- Cuando agreguemos modelos, aparecen aquí automáticamente

Visitar: `http://127.0.0.1:8000/admin/`

**El Admin viene GRATIS con Django — En otros frameworks tendrías que construirlo desde cero**

# Resumen de comandos Django

| Acción | Comando |
|---|---|
| Crear proyecto | `django-admin startproject X` |
| Crear app | `python manage.py startapp X` |
| Aplicar migraciones | `python manage.py migrate` |
| Correr servidor | `python manage.py runserver` |
| Crear superusuario | `python manage.py createsuperuser` |

## Archivos importantes:

| Archivo | Para qué |
|---|---|
| `manage.py` | Ejecutar comandos |
| `settings.py` | Configuración del proyecto |
| `urls.py` | Definir rutas |
| `views.py` | Lógica de las páginas |

# Resumen — Django Setup

**Lo que logramos hoy:**

- Instalamos Django en entorno virtual
- Creamos proyecto “dashboard”
- Entendimos la estructura de archivos
- Corrimos el servidor de desarrollo
- Creamos app “core”
- Hicimos nuestras primeras vistas
- Conocimos el Admin de Django

**PRÓXIMA CLASE**

## URLs y Views

- Sistema de URLs en profundidad
- URLs con parámetros
- Organizar URLs por app
- Más vistas con lógica

**Proyecto Dashboard E-commerce:**

Hoy: Estructura base | Siguiente: Definir rutas